zeus: A Python implementation of Ensemble Slice Sampling for efficient Bayesian parameter inference

Minas Karamanis,[1★] Florian Beutler,[1] and John A. Peacock[1]
[1]Institute for Astronomy, University of Edinburgh, Royal Observatory, Blackford Hill, Edinburgh EH9 3HJ, UK

Accepted XXX. Received YYY; in original form ZZZ

arXiv:2105.03468v2 [astro-ph.IM] 3 Oct 2021

ABSTRACT

We introduce zeus, a well-tested Python implementation of the Ensemble Slice Sampling (ESS) method for Bayesian parameter inference. ESS is a novel Markov chain Monte Carlo (MCMC) algorithm specifically designed to tackle the computational challenges posed by modern astronomical and cosmological analyses. In particular, the method requires only minimal hand–tuning of 1 – 2 hyper-parameters that are often trivial to set; its performance is insensitive to linear correlations and it can scale up to 1000s of CPUs without any extra effort. Furthermore, its locally adaptive nature allows to sample efficiently even when strong non-linear correlations are present. Lastly, the method achieves a high performance even in strongly multimodal distributions in high dimensions. Compared to emcee, a popular MCMC sampler, zeus performs 9 and 29 times better in a cosmological and an exoplanet application respectively.

Key words: methods: statistical – methods: data analysis – techniques: radial velocities – cosmology: large-scale structure of Universe

1 INTRODUCTION

Over the past few decades the volume of astronomical and cosmological data has increased substantially. In response to that, a variety of astrophysical models have been developed to explain the plethora of observations. Markov chain Monte Carlo (MCMC) has been established as the standard procedure of inferring the model parameters subject to the available data in a Bayesian framework. Within the Bayesian context, the object that quantifies the probability distribution of the model parameters θ given the data D and model $\mathcal{M}$ is the posterior distribution $\mathcal{P}(\theta) \equiv P(\theta|D, \mathcal{M})$ which is defined using Bayes's theorem:

$$\mathcal{P}(\theta) = \frac{\mathcal{L}(\theta)\pi(\theta)}{\mathcal{Z}}, \tag{1}$$

where $\mathcal{L}(\theta) \equiv P(D|\theta, \mathcal{M})$ is the likelihood function, $\pi(\theta) \equiv P(\theta|\mathcal{M})$ is the prior distribution of the model parameters θ, and $\mathcal{Z} \equiv P(D|\mathcal{M})$ is the, so called, Bayesian model evidence or marginal likelihood and in this context can be treated as a simple normalisation constant.

MCMC does not in general require knowing the value of the model evidence and it only depends on the ability to evaluate the unnormalised posterior distribution for arbitrary values of θ. MCMC methods can then be used to generate (Markov) chains of samples from the posterior distribution. Those samples can be used to calculate integrals (e.g. parameter uncertainties, marginal distributions etc.) that are paramount for modern astronomical and cosmological analyses.

The most commonly used MCMC methods are variants of the Metropolis-Hastings (MH) algorithm (Metropolis et al. 1953; Hastings 1970). MH consists of two steps. First, given the last sample in the chain, a new sample is proposed and then the Metropolis criterion determines whether or not that new sample should be accepted and thus added to the chain. The resulting chain is Markovian in the sense that each sample is proposed based only on the previous sample. The purpose of the Metropolis acceptance criterion is to bias the chain so that the time spent in a region of the parameter space would be proportional to the posterior probability in that region. In other words, the stationary distribution of the Markov chain is the target distribution i.e. the posterior distribution. For a detailed introduction to MCMC methods we direct the reader to MacKay (2003) and for an intuitive introduction to Bayesian inference to Jaynes (2003).

Arguably, the most difficult part of the MH algorithm is the proposal step. There are many ways of choosing a new sample and the efficiency of the method depends on this choice. By far the simplest one is the use of a normal (Gaussian) distribution, centred around the previous sample to generate the new proposed sample. The resulting method is often called Random Walk Metropolis algorithm and its performance is highly sensitive to the $n(n+1)/2$ elements that form its covariance matrix. Those elements generally need to be chosen *a priori* or be adaptively tuned. More efficient methods utilise the gradient of the target distribution (Betancourt 2017) or an ensemble of parallel and communicating chains (Gilks et al. 1994; Ter Braak 2006; ter Braak & Vrugt 2008; Goodman & Weare 2010).

Out of the methods mentioned in the previous paragraph we will focus our attention on the last one, the ensemble or population MCMC variety. The reason is simple: the Random Walk Metropolis algorithm requires a great amount of tuning (or *a priori* knowledge) for it to perform efficiently and even then there is no guarantee that the proposal covariance matrix is optimal for the whole parameter space. On the other hand, gradient based methods, although very powerful,

★ E-mail: minas.karamanis@ed.ac.uk

© 2021 The Authors

are in general unsuitable for astronomical applications in which the models that are used are almost always not differentiable.

One benefit of ensemble MCMC over its alternatives is that the ensemble of parallel chains (also known as walkers) collectively sample the posterior, thus information about their distribution can be shared and used to make better educated proposals. Other advantages include the lack of hand-tuning of hyper-parameters and their capacity for parallel implementation. For the aforementioned reasons, ensemble MCMC methods have dominated astronomical analyses. The most common ones are affine–invariant ensemble sampling (AIES) (Goodman & Weare 2010) and differential evolution MCMC (DEMC) (Ter Braak 2006; ter Braak & Vrugt 2008), both implemented in the popular `Python` package `emcee` (Foreman-Mackey et al. 2013, 2019).

In this paper we introduce `zeus`, a stable and well-tested `Python` implementation of Ensemble Slice Sampling (ESS) (Karamanis & Beutler 2020). ESS is a method based on the ensemble MCMC paradigm, with the crucial difference being that its proposals are performed via Slice Sampling updates (Neal 2003) instead of Metropolis-Hastings ones. As we will thoroughly demonstrate in Section 3, this subtle difference leads to substantial improvements in terms of sampling efficiency and robustness. `zeus` is a user-friendly tool that does not require any hand-tuning or preliminary runs and can scale up to 1000s of CPUs without any extra effort from the user.

`zeus` has been used in various astronomical and cosmological analyses, including cosmological tests of gravity (Tamosiunas 2020), relativistic effects and primordial non-Gaussianity (Wang et al. 2020), 21cm intensity mapping (Umeh et al. 2021), and has been implemented as part of the `CosmoSIS` package (Zuntz et al. 2015).

`zeus` is open source software that is publicly available at `https://github.com/minaskar/zeus` under the `GPL-3 Licence`. Detailed documentation and examples on how to get started are available at `https://zeus-mcmc.readthedocs.io`.

2 ENSEMBLE SLICE SAMPLING

`zeus` is a `Python` implementation of the Ensemble Slice Sampling (ESS) method presented in Karamanis & Beutler (2020). Here we will provide a high-level description of the method and will refer to the accompanying paper for more details about the underlying algorithmic structure and mathematics.

ESS combines the ensemble MCMC paradigm with slice sampling. Since the use of slice sampling in astronomical parameter inference is rare we will start by explaining its function and how it differs from Metropolis updates. Then we will move on to discuss how it can be efficiently combined with ensemble MCMC.

2.1 Slice sampling

Slice sampling is based on the idea that sampling from a distribution with density $P(x)$ is equivalent to uniform sampling from the area under the plot of $f(x) \propto P(x)$. To this end, we introduce an auxiliary variable y, called height, such that the joint distribution $P(x, y)$ is uniform over the region $U = \{(x, y) : 0 < y < f(x)\}$. To sample from the marginal distribution $P(x)$, we first sample from $P(x, y)$ and then we marginalise by dropping the y value of each sample.

In order to generate samples from $P(x, y)$ we utilise the following scheme (Neal 2003):

(i) Given the current state x_0, draw y_0 uniformly from $(0, f(x_0))$.

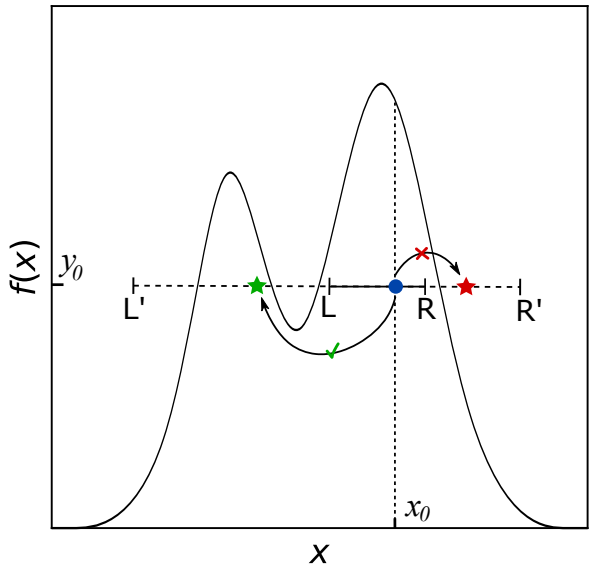

Figure 1. Illustration of the univariate slice sampling update. Given the current sample x_0, a value y_0 is uniformly sampled along the vertical slice $(0, f(x_0))$ (dashed line) thus defining the initial point (blue). An interval (L, R) is uniformly positioned horizontally around (x_0, y_0) and it is expanded in steps of size $R - L$ until both its ends are outside the slice. The new sample is generated by repeatedly sampling (uniformly) from the interval (L', R') until a sample (green star) is found inside the slice. Samples outside of the slice (red star) are rejected and they are instead used to shrink (L', R').

(ii) Find an interval $I = (L, R)$ that contains all, or at least part, of the slice $s = \{x : y_0 < f(x)\}$.

(iii) Draw the new sample x_1 uniformly from $I \cap S$.

To construct the interval I (step ii), Neal (2003) introduced the stepping-out procedure that works by randomly positioning an interval of length μ around the sample x_0 (i.e. blue dot in Figure 1) and then expanding it in steps of size μ until both its ends (i.e. L' and R') are outside the slice. To obtain x_1 (i.e. green star in Figure 1) we then use the shrinking procedure in which candidates are sampled uniformly from I until a point inside the slice S is found. Samples outside of the slice are used to shrink the interval I. The two procedures are shown in Figure 1.

The length scale μ is the only free hyperparameter of slice sampling and although its choice can reduce or increase the computational cost of the method it generally does not affect its mixing properties (e.g. convergence rate, autocorrelation time, etc.). `zeus` utilises a stochastic optimization algorithm similar to Tibbits et al. (2014) and based on the Robbins & Monro (1951) optimisation scheme in order to tune μ to its optimal value (see Section 3.1 of Karamanis & Beutler 2020 for more details).

It is important to note here that for multimodal target distributions there is no guarantee that the approximate slice would cross any of the other modes. In particular, if the initial estimate of the length scale μ is low then the probability of missing the other peaks, assuming that they are located far away, is also low. As we will show in Section 3, unlike simple slice sampling, ESS and thus `zeus` does not suffer from this effect.

2.2 Walkers, moves and parallelism

The slice sampling update described in the previous paragraphs is a univariate update scheme. For it to be used to sample from mul-

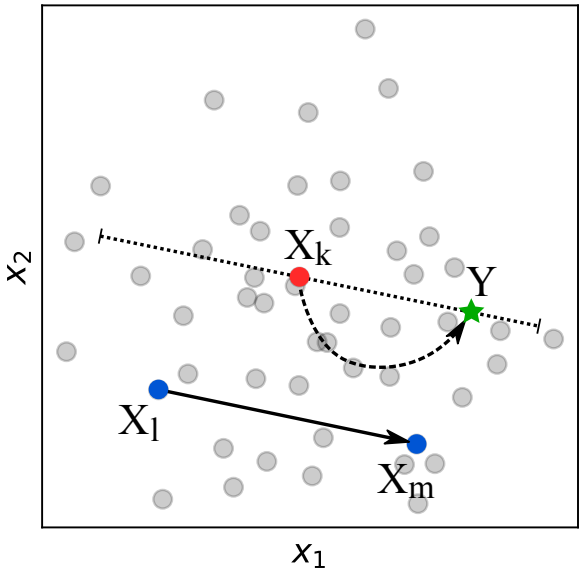

Figure 2. The figure illustrates the differential move in the context of Ensemble Slice Sampling. The walker X_k to be updated is shown in red. Two walkers, X_l and X_m, (blue) are uniformly selected from the complementary ensemble (grey). The approximate slice (dotted line) is constructed parallel to the two walkers X_l and X_m using the stepping-out procedure. The new position Y (green) of X_k is sampled using the shrinking procedure along the approximate slice.

tivariate target distributions it needs to be generalised accordingly. Perhaps the simplest such generalisation in a multivariate setting is the use of slice sampling to sample along each coordinate axis in turn (i.e. component-wise slice sampling) or to sample along randomly selected directions in parameter space (MacKay 2003). Although valid, both of these approaches are unsuitable in cases of correlated parameters in which the proper choice of direction can substantially accelerate mixing.

To address this issue, Tibbits et al. (2014) proposed to orthogonalise the parameter space using the sample covariance, thus getting rid of linear correlations between parameters. We will instead follow a different, perhaps more flexible, approach to construct an efficient slice sampler. Our aim is to utilise an ensemble of parallel chains/walkers that can exchange information about the covariance structure of the target distribution and thus by-pass the difficulties posed by correlations.

As hinted in the introduction, the ensemble of walkers collectively sample the target distribution and thus their positions encode information about the correlations between the parameters. One way to take advantage of this information is to use it to construct direction vectors along which slice sampling can take place. Many moves that generate direction vectors from the complementary ensemble are possible. `zeus` offers a collection of them, including some that utilise clustering algorithms and density estimation methods. As we will show in Section 3, such moves can help accelerate sampling in difficult cases such as strongly multimodal distributions. Any distribution of the complementary ensemble can be used as a valid proposal to generate such direction vectors and `zeus` offers a highly flexible interface for the user to define such a move or choose one (or a mixture) from the ones that are already implemented and tested. Here is a list of the currently implemented moves in `zeus`:

- **Differential move:** This is the default move used by `zeus` and shown in Figure 2. Using the differential move, Ensemble Slice Sampling updates the position of each walker in the ensemble by slice sampling along a direction defined by the difference between two uniformly selected walkers from the rest of the ensemble (i.e. the complementary ensemble).
- **Gaussian move:** The Gaussian move samples the direction vectors along which slice sampling is performed from a normal distribution that shares the same covariance structure as the complementary ensemble. This approach is very efficient in cases in which the target distribution is close to normal.
- **Global move:** The Global move utilises a Dirichlet Process Gaussian Mixture to fit the complementary ensemble and proposes directions along different peaks of the target distribution in cases of strong multi-modality.
- **KDE move:** The KDE move samples the direction vectors from a Gaussian Kernel Density Estimate of the complementary ensemble. This can be useful in cases of highly non-Gaussian target distributions.
- **Random move:** The Random move performs slice sampling along isotropic directions. This is equivalent of standard multivariate slice sampling and it is mostly offered for testing purposes as it cannot handle correlations efficiently.

For more information on how those moves work as well as a comparison of the Differential, Gaussian and Global moves we direct the interested reader to Karamanis & Beutler (2020). Unless stated otherwise the Differential move will be used for the following examples.

To parallelise this process and capitalise on the availability of multiple CPUs we randomly split the ensemble into two sets of walkers (i.e. active and passive sets) (Foreman-Mackey et al. 2013) and choose to update the positions of the active walkers along direction vectors defined by passive walkers. Then the passive become active and *vice versa* and the process is repeated. The ensemble splitting technique is required in order to parallelise the algorithm without violating detailed balance. Parallelisation is achieved in practise using either `multiprocessing` or `MPI` using the implemented `ChainManager` utility that can distribute both multiple ensembles and multiple chains in parallel computing environments at the same time. Heuristics to determine the number of required walkers per application are discussed in Section 4.

3 EMPIRICAL EVALUATION

For the empirical evaluation of `zeus` we use five toy examples that manifest significant aspects of real astronomical applications[1] (i.e. linear and non-linear correlations, multimodality, heavy tails, hard boundaries) and two real-world astronomical examples characteristic of modern astronomical analyses.

3.1 Toy examples

In order to understand the behaviour of `zeus` in various sampling scenarios it is important to study its performance in different toy examples that demonstrate different characteristics of common target distributions that arise in astronomical applications. For that reason we chose five such toy examples. The first one is a normal (Gaussian)

[1] For additional demonstrations on similarly common structures (e.g. the funnel) we direct the reader to the accompanying paper (Karamanis & Beutler 2020).